Assignment 5

Server.cpp

```
%%writefile server.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 45678
#define BUFFER_SIZE 1024

int main()
{
    int server_socket, client_socket;
    struct sockaddr_in server_address, client_address;
    socklen_t client_length;

    char buffer[BUFFER_SIZE];

    // 1. Create TCP socket
    server_socket = socket(AF_INET, SOCK_STREAM, 0);

    if (server_socket < 0)
    {
        perror("Socket creation failed");
        return 1;
    }

    printf("Server socket created successfully.\n");

    // 2. Configure server address
    server_address.sin_family = AF_INET;
    server_address.sin_addr.s_addr = INADDR_ANY;
    server_address.sin_port = htons(PORT);
```

```
// 3. Bind socket
if (bind(server_socket,
         (struct sockaddr *)&server_address,
         sizeof(server_address)) < 0)
{
    perror("Bind failed");
    close(server_socket);
    return 1;
}

printf("Socket bound to port %d.\n", PORT);

// 4. Listen for client
if (listen(server_socket, 5) < 0)
{
    perror("Listen failed");
    close(server_socket);
    return 1;
}

printf("Server is listening...\n");

// 5. Accept client connection
client_length = sizeof(client_address);

client_socket = accept(server_socket,
                       (struct sockaddr *)&client_address,
                       &client_length);

if (client_socket < 0)
{
    perror("Accept failed");
    close(server_socket);
    return 1;
}

printf("Client connected successfully.\n");

// 6. Receive message from client
```

```
    memset(buffer, 0, BUFFER_SIZE);

    recv(client_socket, buffer, BUFFER_SIZE - 1, 0);

    // 7. Display received message
    printf("Message received from client: %s\n", buffer);

    // 8. Send reply to client
    char reply[] = "Hello Client! Message received successfully.";

    send(client_socket, reply, strlen(reply), 0);

    printf("Reply sent to client.\n");

    // 9. Close client connection
    close(client_socket);

    // 10. Close server socket
    close(server_socket);

    printf("Server closed.\n");

    return 0;
}
```

Client.cpp

```
%%writefile client.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 45678
#define BUFFER_SIZE 1024

int main()
{
```

```
int client_socket;
struct sockaddr_in server_address;

char message[BUFFER_SIZE];
char buffer[BUFFER_SIZE];

// 1. Create TCP socket
client_socket = socket(AF_INET, SOCK_STREAM, 0);

if (client_socket < 0)
{
    perror("Socket creation failed");
    return 1;
}

printf("Client socket created successfully.\n");

// 2. Configure server address
server_address.sin_family = AF_INET;
server_address.sin_port = htons(PORT);

// Server is running on localhost
if (inet_pton(AF_INET, "127.0.0.1",
              &server_address.sin_addr) <= 0)
{
    perror("Invalid address");
    close(client_socket);
    return 1;
}

// 3. Connect to server
if (connect(client_socket,
            (struct sockaddr *)&server_address,
            sizeof(server_address)) < 0)
{
    perror("Connection failed");
    close(client_socket);
    return 1;
}
```

```
    printf("Connected to server.\n");

    // 4. Input message from user
    printf("Enter message for server: ");
    fgets(message, BUFFER_SIZE, stdin);

    // Remove newline
    message[strcspn(message, "\n")] = '\0';

    // 5. Send message to server
    send(client_socket, message, strlen(message), 0);

    printf("Message sent to server.\n");

    // 6. Receive reply from server
    memset(buffer, 0, BUFFER_SIZE);

    recv(client_socket, buffer, BUFFER_SIZE - 1, 0);

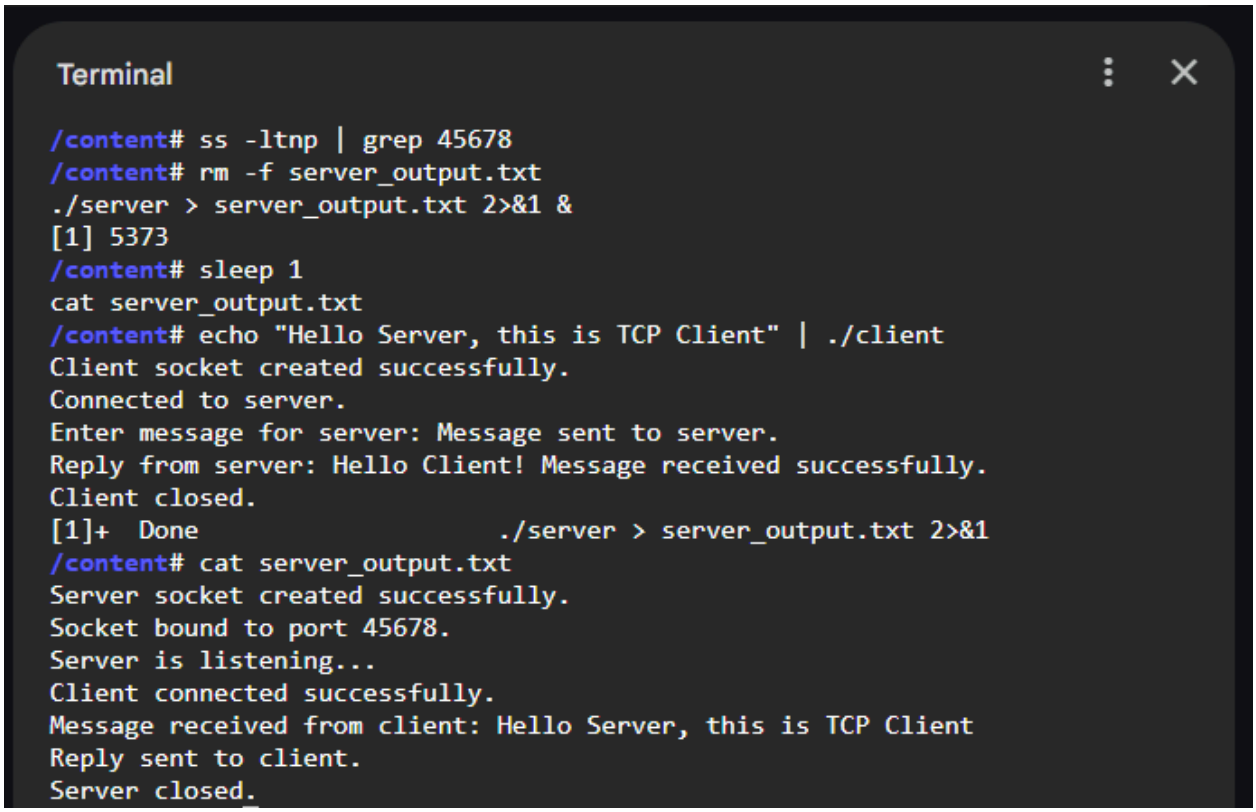
    // 7. Display server reply
    printf("Reply from server: %s\n", buffer);

    // 8. Close client socket
    close(client_socket);

    printf("Client closed.\n");

    return 0;
}
```

OUTPUT


Terminal
/content# ss -ltnp | grep 45678
/content# rm -f server_output.txt
./server > server_output.txt 2>&1 &
[1] 5373
/content# sleep 1
cat server_output.txt
/content# echo "Hello Server, this is TCP Client" | ./client
Client socket created successfully.
Connected to server.
Enter message for server: Message sent to server.
Reply from server: Hello Client! Message received successfully.
Client closed.
[1]+ Done ./server > server_output.txt 2>&1
/content# cat server_output.txt
Server socket created successfully.
Socket bound to port 45678.
Server is listening...
Client connected successfully.
Message received from client: Hello Server, this is TCP Client
Reply sent to client.
Server closed.